UNIDAD DE TRABAJO 1 (PARTE 2)

UT1: Manejo de ficheros y persistencia en Java

Manipulación avanzada de ficheros, lectura/escritura en UTF-8, copia, movimiento atómico y recorridos recursivos de directorios e inspección de metadatos,.

Comparativa de arquitecturas: java.io vs. java.nio.file

API clásica (java.io.File)

- Introducida en Java 1.0 basada en la clase File.
- Objeto mutable que agrupa ruta y operaciones.
- Devuelve booleanos (true/false) sin indicar la causa real del fallo.
- Sin soporte nativo para copia/movimiento directo ni eventos de disco.
- Lectura de directorios no recursiva (listFiles devuelve null si no puede listar el contenido).

API moderna (java.nio.file / NIO.2)

- Introducida en Java 7 (NIO.2) con diseño desacoplado (separa la representación de la ruta, las operaciones sobre archivos y el mecanismo que accede al sistema de archivo.)
- Path (interfaz inmutable) representa la localización del recurso.
- Files (clase de utilidades estáticas) ejecuta las operaciones.
- Gestión robusta de errores mediante jerarquía de IOException.
- Soporta movimiento atómico de ficheros, observadores (WatchService) y Streams perezosos.

Estrategias de lectura de texto plano y conjuntos de caracteres

Métodos directos de NIO.2

- Files.readString(path, charset): Carga todo el texto en un String. Ideal para ficheros pequeños.
- Files.readAllLines(path, charset): Devuelve List<String>. Permite recorrer líneas en RAM.
- Files.lines(path, charset): Devuelve Stream<String> perezoso. Procesa archivos gigantes sin saturar memoria.
- Siempre fijar StandardCharsets.UTF-8 para evitar caracteres corruptos.

Enfoque tradicional java.io

- BufferedReader + InputStreamReader: Permite fijar codificación UTF-8 explícita sobre FileInputStream.
- FileReader sin parámetros desaconsejado: Utiliza la codificación por defecto del SO.
- Uso de readLine() en bucle while hasta recibir null.
- Obligatoriedad de cierre dentro de bloque try-with-resources.

Lectura completa y por lista con NIO.2

```
// Lectura completa con Files.readString
Path ruta = Path.of("datos/mensaje.txt");

try {
    // Lectura directa en UTF-8 a un único String
    String contenido = Files.readString(ruta,
StandardCharsets.UTF_8);
    System.out.println("=== Contenido ===");
    System.out.println(contenido);
} catch (IOException e) {
    System.err.println("Error al leer: " +
e.getMessage());
}
```

```
// Lectura a lista con Files.readAllLines
Path ruta = Path.of("datos/alumnos.txt");

try {
    // Lectura de todas las líneas a una lista
    List<String> lineas = Files.readAllLines(ruta,
StandardCharsets.UTF_8);
    System.out.println("Total líneas: " +
lineas.size());
    for (int i = 0; i < lineas.size(); i++) {
        System.out.println("Línea " + (i + 1) + ": "
+ lineas.get(i));
    }
} catch (IOException e) {
    System.err.println("Error al leer lista: " +
e.getMessage());
}
```

Lectura perezosa con Stream y java.io

```
// Lectura perezosa con Stream
    Path ruta = Path.of("datos/log_gigante.log");

// Stream dentro del try-with-resources para
// cerrar el canal
try (Stream<String> lineas = Files.lines(ruta,
StandardCharsets.UTF_8)) {
    lineas.filter(linea -> linea.contains("ERROR"))
          .forEach(linea -> System.out.println("-> "
+ linea));
} catch (IOException e) {
    System.err.println("Error en stream: " +
e.getMessage());
}
```

```
// Lectura clásica en UTF-8
File archivo = new File("datos/notas.txt");

try (FileInputStream fis = new
FileInputStream(archivo);
     InputStreamReader isr = new
InputStreamReader(fis, StandardCharsets.UTF_8);
     BufferedReader br = new BufferedReader(isr)) {

    String linea;
    while ((linea = br.readLine()) != null) {
        System.out.println("> " + linea);
    }
} catch (IOException e) {
    System.err.println("Error lectura clásica: " +
e.getMessage());
}
```

Escritura de texto plano y opciones de apertura de ficheros

Escritura moderna en NIO.2

- Files.writeString(path, text, charset, options): Escritura directa de cadenas.
- Files.write(path, iterable, charset, options): Escritura de listas de líneas.
- Crea automáticamente directorios intermedios necesarios combinando con createDirectories.
- Sobrescribe el contenido por defecto si no se especifican opciones de anexado.

StandardOpenOption

- CREATE: Crea el fichero si no existe.
- APPEND: Anexa el contenido al final del fichero existente.
- CREATE_NEW: Crea el fichero, lanzando FileAlreadyExistsException si ya existía.
- TRUNCATE_EXISTING: Vacía el contenido previo si el fichero ya existe.

Escritura, anexado y formato con PrintWriter

```
// Anexado con StandardOpenOption
Path ruta = Path.of("salida/traza.log");

try {
    Files.createDirectories(ruta.getParent());
    String log = "INFO [" +
System.currentTimeMillis() + "] Evento registrado
";

    // Opciones CREATE y APPEND combinadas
    Files.writeString(ruta, log,
StandardCharsets.UTF_8, CREATE, APPEND);
    System.out.println("Entrada anexada con
éxito.");
} catch (IOException e) {
    System.err.println("Error al anexar: " +
e.getMessage());
}
```

```
// Formato con PrintWriter en UTF-8
File archivo = new File("salida/factura.txt");
archivo.getParentFile().mkdirs();

try (FileOutputStream fos = new
FileOutputStream(archivo, false);
     OutputStreamWriter osw = new
OutputStreamWriter(fos, StandardCharsets.UTF_8);
     PrintWriter pw = new PrintWriter(osw)) {

    pw.println("=== FACTURA DE COMPRA ===");
    pw.printf("Producto: %-15s | Precio: %6.2f €%n",
"Teclado RGB", 45.99);
    System.out.println("Reporte generado.");
} catch (IOException e) {
    System.err.println("Error en factura: " +
e.getMessage());
}
```

Copia de archivos, duplicación de metadatos y copia recursiva

Copia directa con NIO.2

- Files.copy(origen, destino, opciones): Copia directa de ficheros.
- StandardCopyOption.REPLACE_EXISTING: Sobrescribe el destino si ya existe.
- StandardCopyOption.COPY_ATTRIBUTES: Preserva marcas temporales y permisos.
- No realiza copia recursiva de carpetas por defecto; solo crea el directorio vacío.

Copia recursiva de directorios

- Combina Files.walk(origen) con la resolución de rutas relativas mediante relativize().
- Si el nodo explorado es un directorio, ejecuta Files.createDirectories.
- Si es un fichero regular, ejecuta Files.copy para trasvasar el contenido.
- Garantiza la duplicación íntegra de árboles de archivos complejos.

Copia de ficheros y copia recursiva

```
// Copia de un fichero con metadatos
Path origen = Path.of("datos/documento.pdf");
Path destino =
Path.of("backup/documento_copia.pdf");

try {
    Files.createDirectories(destino.getParent());
    // Copiar reemplazando y conservando metadatos
    Files.copy(origen, destino, REPLACE_EXISTING,
COPY_ATTRIBUTES);
    System.out.println("Copia realizada con
éxito.");
} catch (IOException e) {
    System.err.println("Error al copiar: " +
e.getMessage());
}
```

```
// Copia recursiva con Files.walk
Path origDir = Path.of("proyecto");
Path destDir = Path.of("copia/proyecto");

try (Stream<Path> arbol = Files.walk(origDir)) {
    arbol.forEach(origen -> {
        try {
            Path destino =
destDir.resolve(origDir.relativize(origen));
            if (Files.isDirectory(origen))
Files.createDirectories(destino);
            else Files.copy(origen, destino,
REPLACE_EXISTING, COPY_ATTRIBUTES);
        } catch (IOException e) { throw new
RuntimeException(e); }
    });
}
```

Mover, renombrar y eliminar ficheros de forma segura

Movimiento atómico (ATOMIC_MOVE)

- Files.move(origen, destino, opciones): Mueve o renombra recursos.
- StandardCopyOption.ATOMIC_MOVE: Transacción indivisible en el sistema de archivos.
- Evita que otros procesos lean ficheros corruptos a medio mover.
- File.renameTo en java.io no es fiable entre diferentes particiones.

Eliminación segura

- Files.delete(path): Elimina el fichero, lanzando NoSuchFileException si no existe.
- Files.deleteIfExists(path): Devuelve true/false evitando excepciones si no existía.
- Borrado de directorios: Requiere que la carpeta esté completamente vacía (DirectoryNotEmptyException).
- in.transferTo(out) en Java 9+: Copia eficiente directa entre flujos clásicos.

Movimiento atómico, borrado seguro y transferTo

// Movimiento atómico (Ejemplo 4.3)

```
Path origen = Path.of("datos/temporal.tmp");
Path destino = Path.of("procesados/definitivo.dat");

try {
    Files.createDirectories(destino.getParent());
    // Transacción indivisible
    Files.move(origen, destino, ATOMIC_MOVE,
REPLACE_EXISTING);
    System.out.println("Archivo movido
atómicamente.");
} catch (IOException e) {
    System.err.println("Error al mover: " +
e.getMessage());
}
```

// Copia por flujos con transferTo (Ejemplo 4.5)

```
File origen = new File("datos/imagen.png");
File destino = new File("backup/imagen.png");

try (InputStream in = new BufferedInputStream(new
FileInputStream(origen));
     OutputStream out = new BufferedOutputStream(new
FileOutputStream(destino))) {

    // Transferencia eficiente directa introducida
en Java 9+
    in.transferTo(out);
    System.out.println("Copia realizada con
transferTo.");
} catch (IOException e) { System.err.println("Error:
" + e.getMessage()); }
```

Listado, filtrado y recorrido de directorios

Exploración en NIO.2

- Files.list(path): Devuelve Stream<Path> con los elementos directos del nivel actual.
- Files.walk(path, maxDepth): Explora recursivamente en profundidad el árbol de ficheros.
- Files.find(path, depth, matcher): Búsqueda filtrada por metadatos (tamaño, fecha).
- PathMatcher Glob (glob:**/*.txt): Filtrado ágil de rutas por comodines.

Exploración en java.io

- dir.listFiles(): Devuelve un array File[] con los elementos directos.
- Control obligatorio de null por si el directorio no existe o falta permisos.
- FilenameFilter: Interfaz para filtrar nombres mediante expresiones lambda.
- Recorrido recursivo: Exige implementar un algoritmo manual de profundización (DFS).

Listado directo y filtrado por sintaxis Glob

```
// Listado del nivel actual con Files.list
Path directorio = Path.of("datos");

// Listado no recursivo del nivel actual
try (Stream<Path> contenido =
Files.list(directorio)) {
    contenido.forEach(elem ->
System.out.println("Elemento: " +
elem.getFileName()));
} catch (IOException e) {
    System.err.println("Error al listar: " +
e.getMessage());
}
```

```
// Filtrado recursivo con patrones Glob
Path raiz = Path.of("proyectos");
// Patrón Glob para buscar .txt en cualquier
profundidad
PathMatcher matcher =
FileSystems.getDefault().getPathMatcher("glob:**/*.t
xt");

try (Stream<Path> arbol = Files.walk(raiz)) {
    arbol.filter(p -> Files.isRegularFile(p) &&
matcher.matches(p))
         .forEach(txt -> System.out.println("TXT: "
+ txt.toAbsolutePath()));
} catch (IOException e) { System.err.println("Error:
" + e.getMessage()); }
```

Búsqueda por predicados y recorrido DFS (Depth First Search)

// Búsqueda por metadatos con Files.find

```
Path raiz = Path.of("servidor/logs");
long limiteBytes = 1024 * 1024; // 1 MB

// Files.find filtra por atributos en tiempo de
exploración
try (Stream<Path> resultados = Files.find(
        raiz, Integer.MAX_VALUE,
        (path, attrs) -> attrs.isRegularFile() &&
attrs.size() >= limiteBytes
)) {
    resultados.forEach(f -> System.out.println("Log
pesado: " + f.getFileName()));
} catch (IOException e) { System.err.println("Error:
" + e.getMessage()); }
```

// Recorrido recursivo manual DFS en java.io

```
public static void explorar(File nodo) {
    File[] hijos = nodo.listFiles();
    if (hijos == null) return; // Control de
seguridad

    for (File f : hijos) {
        if (f.isDirectory()) {
            System.out.println("[DIR]  " +
f.getAbsolutePath());
            explorar(f); // Llamada recursiva DFS
        } else { System.out.println("  [FILE] " +
f.getName()); }
    }
}
```

Inspección de metadatos y vistas de atributos de archivos

Concepto de metadato

- Datos que describen las propiedades del archivo sin ser el contenido real.
- Incluyen nombre, tamaño, fechas, permisos y propietario.
- Fundamentales para la gestión del sistema de archivos y seguridad.

BasicFileAttributes

- Lectura en bloque de atributos multiplataforma en una sola llamada.
- creationTime(): Fecha de creación.
- lastModifiedTime(): Última edición.
- size(): Tamaño en bytes.
- isDirectory() e isRegularFile(): Tipo de recurso.

PosixFileAttributes

- Específico de sistemas Unix, Linux y macOS.
- owner(): Usuario propietario.
- group(): Grupo del sistema.
- permissions(): Permisos de acceso POSIX (rwx).
- Permite gestionar la seguridad de archivos en servidores Linux.

Consulta de metadatos con NIO.2 y java.io

// Movimiento atómico de un fichero

```
import java.nio.file.*;
...
Path origen = Path.of("datos/archivo.txt");
Path destino =Path.of("copia/archivo.txt");
Files.move(origen, destino,
StandardCopyOption.ATOMIC_MOVE);
```

// Uso de Watchservice

```
import java.nio.file.*;

public class VigilarCarpeta {
    public static void main(String[] args) throws Exception {

        Path carpeta = Path.of("documentos");

        WatchService vigilante =
                FileSystems.getDefault().newWatchService();
        // Indicamos qué cambios queremos detectar
        carpeta.register(
                vigilante,
                StandardWatchEventKinds.ENTRY_CREATE
        );
        System.out.println("Vigilando la carpeta documentos...");

        // Esperamos a que ocurra un cambio
        WatchKey clave = vigilante.take();
        // Mostramos los archivos creados
        for (WatchEvent<?> evento : clave.pollEvents()) {
```

Consulta de metadatos con NIO.2 y java.io

```
// Metadatos en bloque con NIO.2
BasicFileAttributes attrs =
Files.readAttributes(ruta,
BasicFileAttributes.class);

System.out.println("Creación: " +
attrs.creationTime());
System.out.println("Modificación: " +
attrs.lastModifiedTime());
System.out.println("¿Es directorio?: " +
attrs.isDirectory());
System.out.println("Tamaño bytes: " + attrs.size());
```

```
// Metadatos con java.io.File
File archivo = new File("datos/reporte.txt");

System.out.println("Ruta absoluta: " +
archivo.getAbsolutePath());
System.out.println("¿Existe?: " + archivo.exists());
System.out.println("¿Se puede leer?: " +
archivo.canRead());

long ms = archivo.lastModified();
String fecha = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss").format(new Date(ms));
System.out.println("Última modificación: " + fecha);
```

Decálogo de buenas prácticas

Buenas prácticas en el uso de ficheros

- Priorizar siempre la API moderna java.nio.file sobre java.io.
- Fijar explícitamente StandardCharsets.UTF-8 en toda lectura/escritura de texto.
- Cerrar siempre los Streams de Files.lines(), Files.walk() y Files.find() en try-with-resources.
- Utilizar ATOMIC_MOVE para renombrar/mover archivos críticos en producción.
- Comprobar valores null en listFiles() de la API clásica.